Crossplane Composition Best Practices

What we learned composing real AWS infrastructure at Gong

in-cluster render | observe | dependencies | ArgoCD health | ready | crossplane render

Daniel Rubin · DevOps, Gong



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

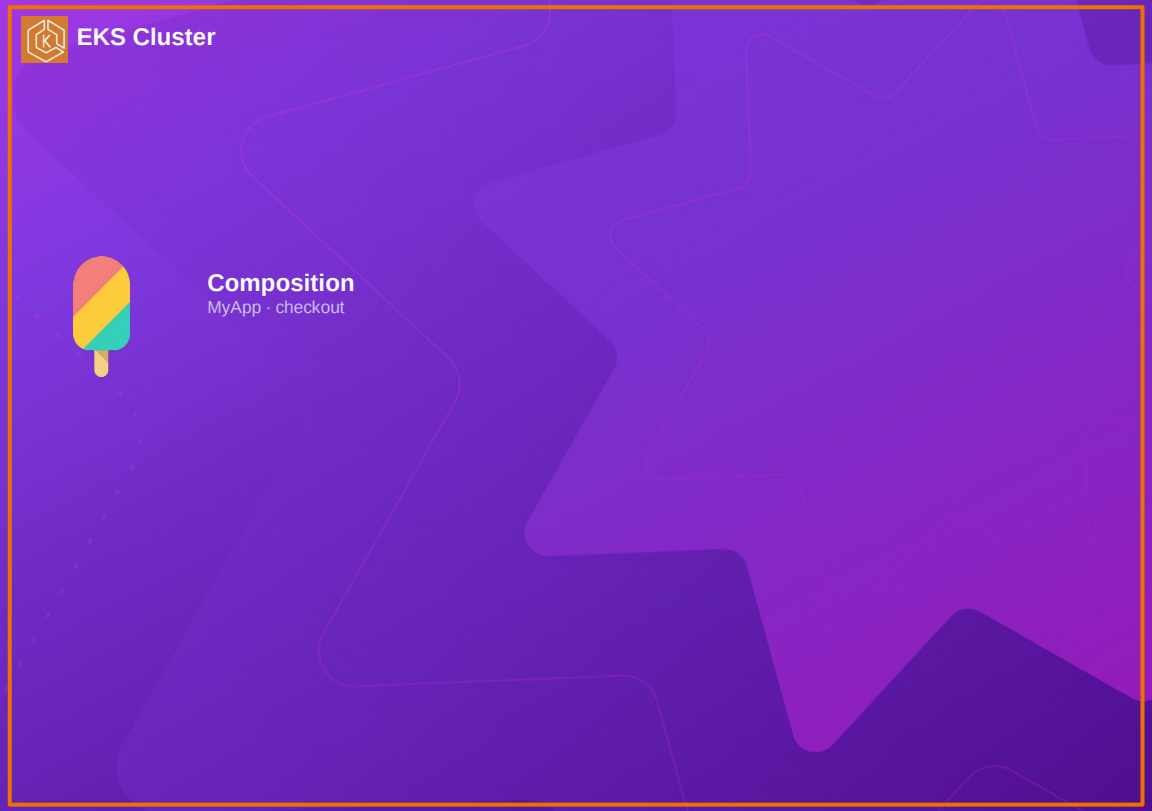
THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.

Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.




```
$ kubectl describe rolepolicyattachment checkout

Warning  CannotResolveResourceReferences
cannot resolve references:
mg.Spec.ForProvider.PolicyArn: referenced
field was empty (referenced resource may
not yet be ready)
```

Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



…errors and retries with back-off until the Role and Policy exist

```
$ kubectl describe rolepolicyattachment checkout

Warning  CannotResolveResourceReferences
cannot resolve references:
mg.Spec.ForProvider.PolicyArn: referenced
field was empty (referenced resource may
not yet be ready)
```

Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



…errors and retries with back-off until the Role and Policy exist

```
$ kubectl describe rolepolicyattachment checkout

Warning  CannotResolveResourceReferences
cannot resolve references:
mg.Spec.ForProvider.PolicyArn: referenced
field was empty (referenced resource may
not yet be ready)
```

What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.

Challenge 2 · Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.



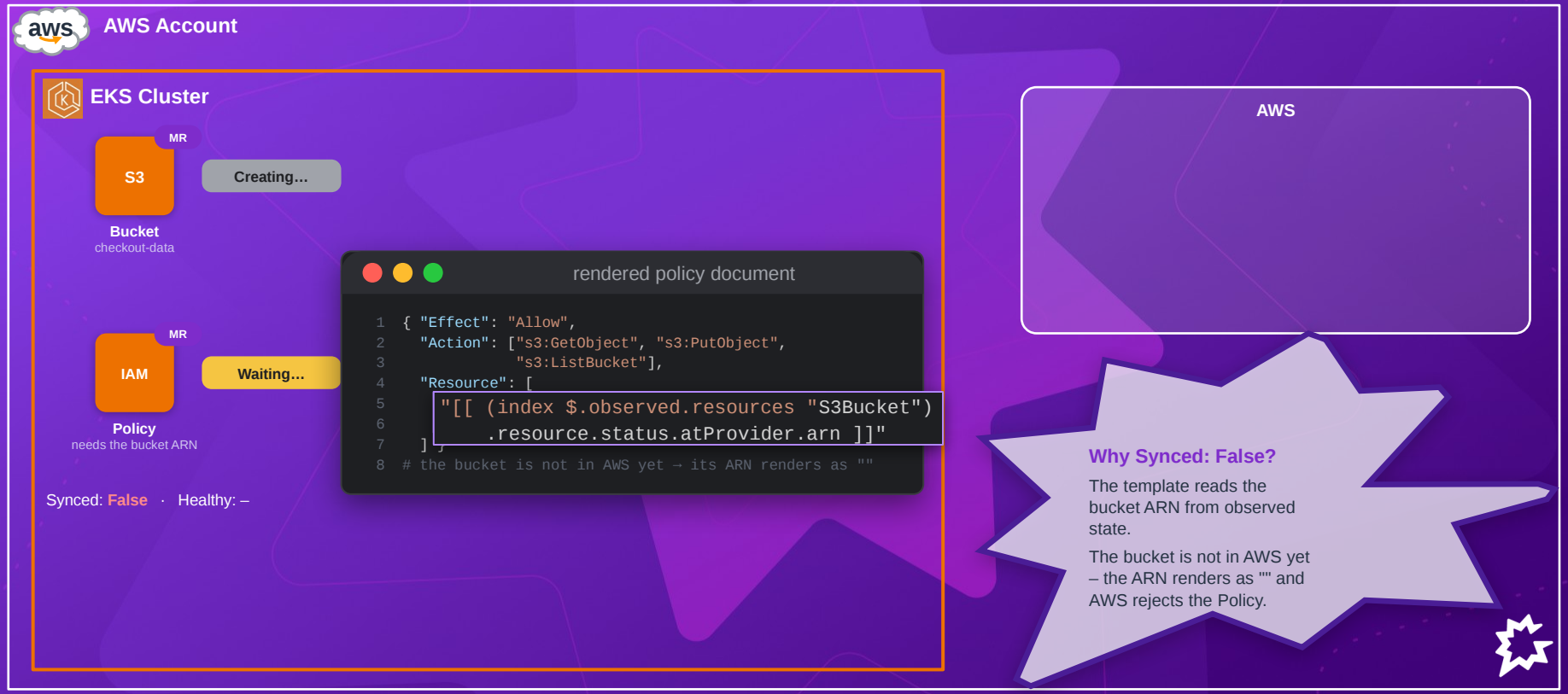
Challenge 2 · Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.



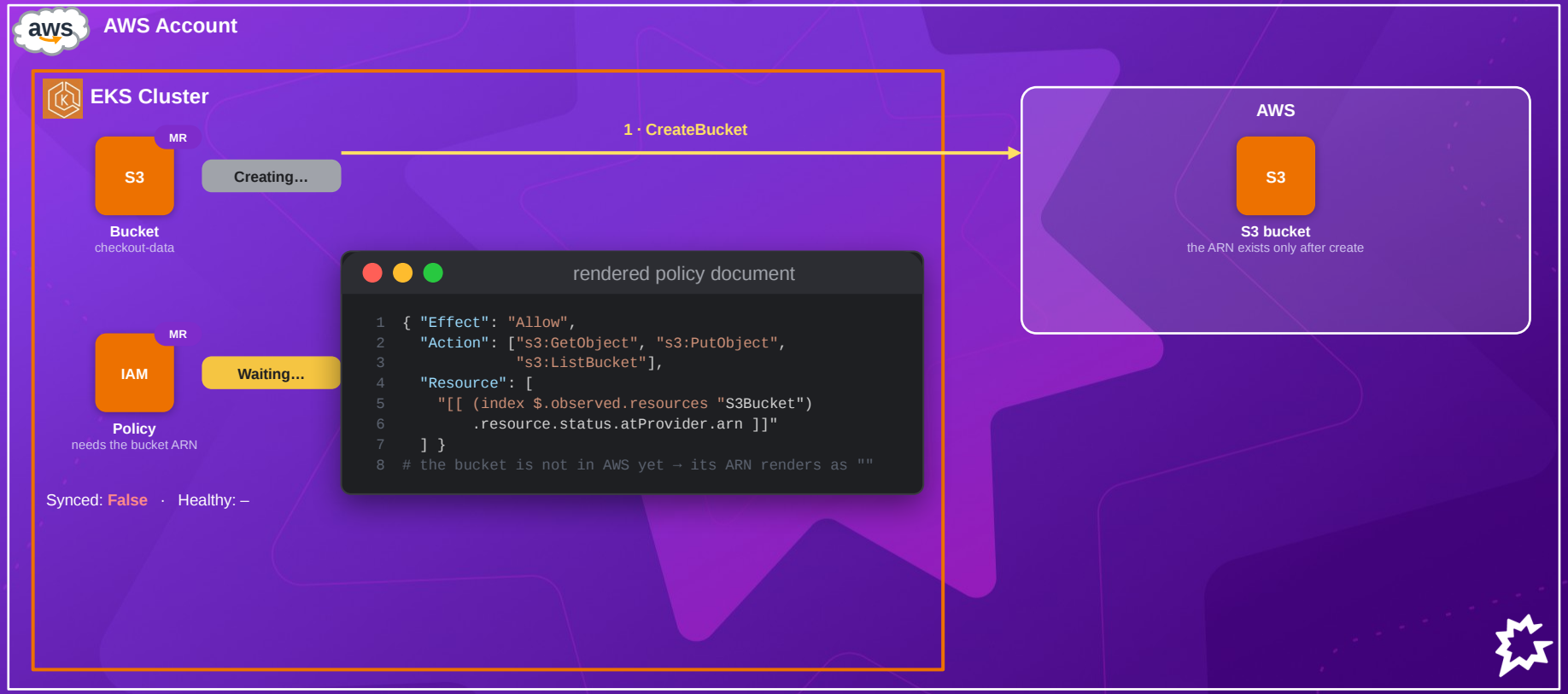
Challenge 2 · Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.



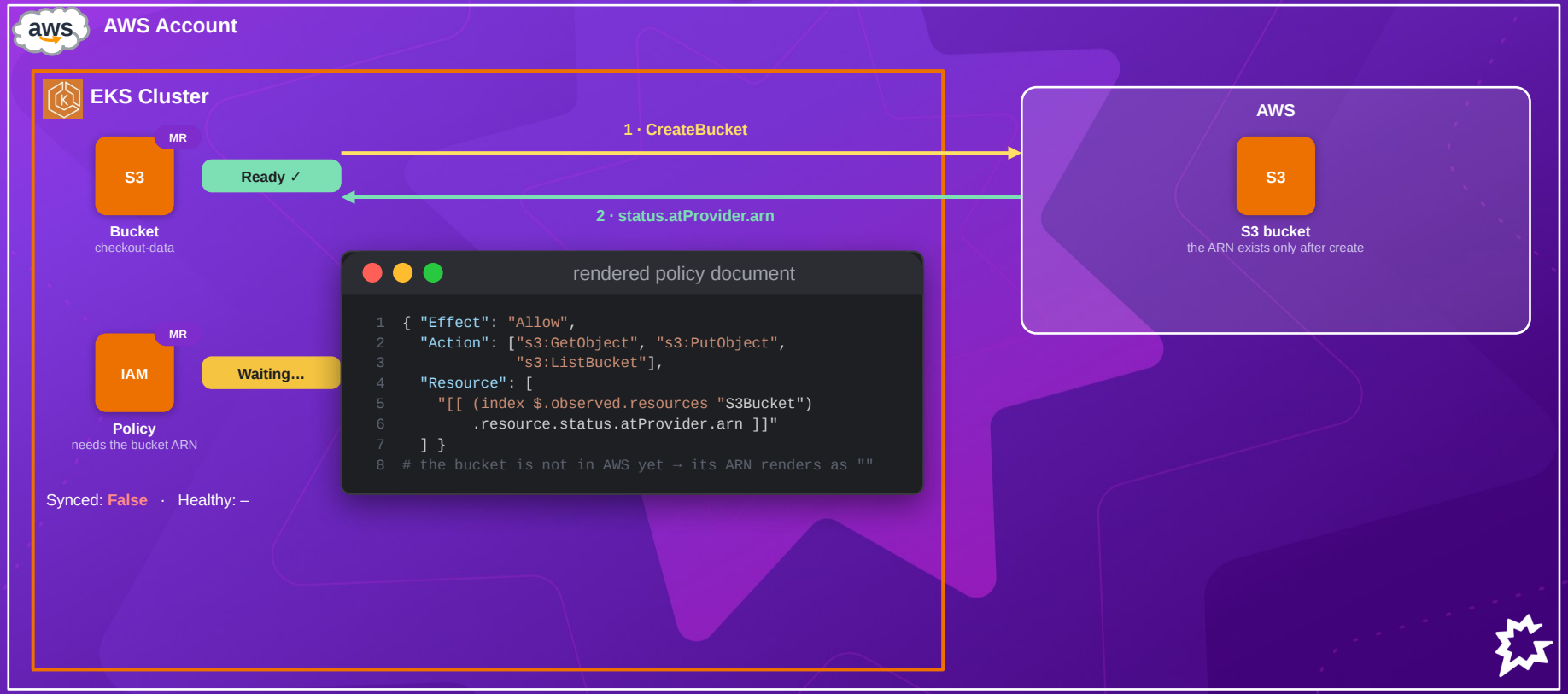
Challenge 2 · Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.



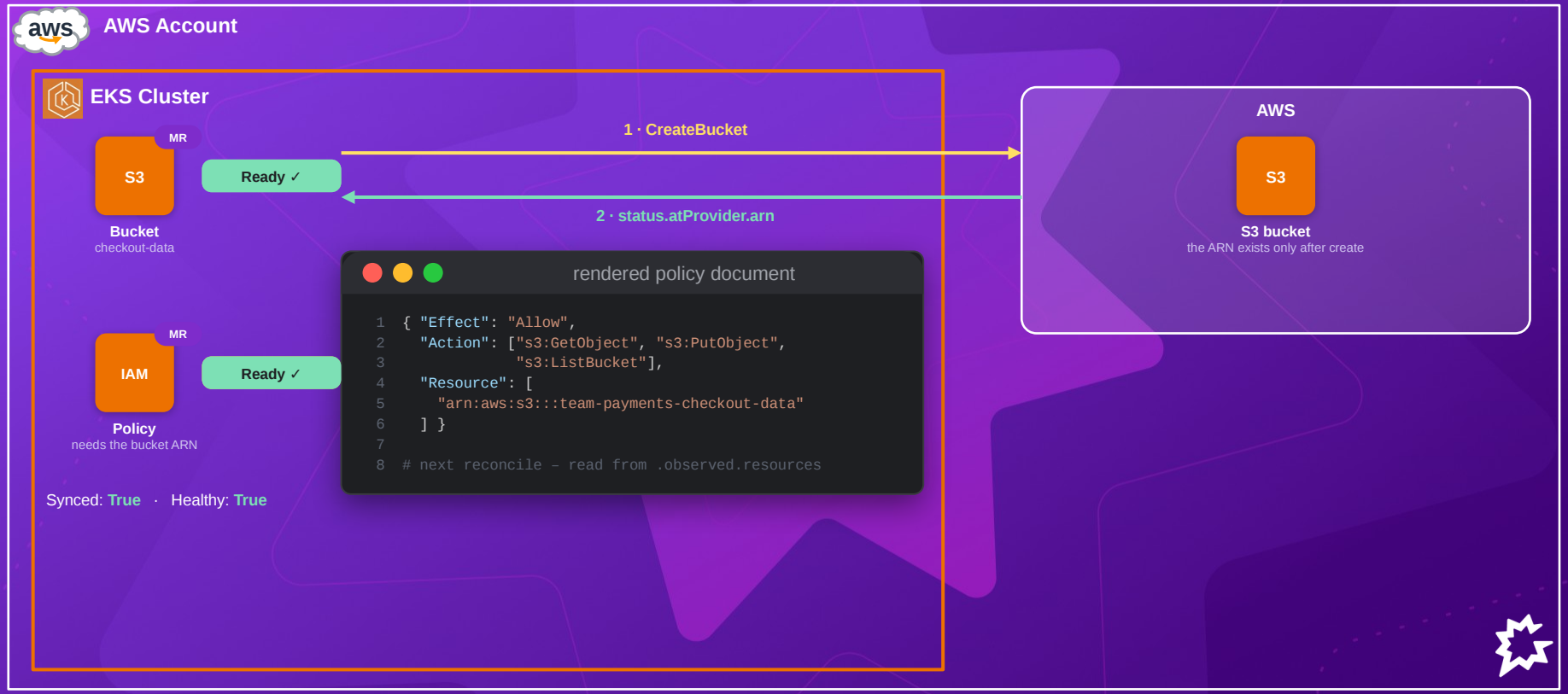
Challenge 2 · Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.



Challenge 2 · Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

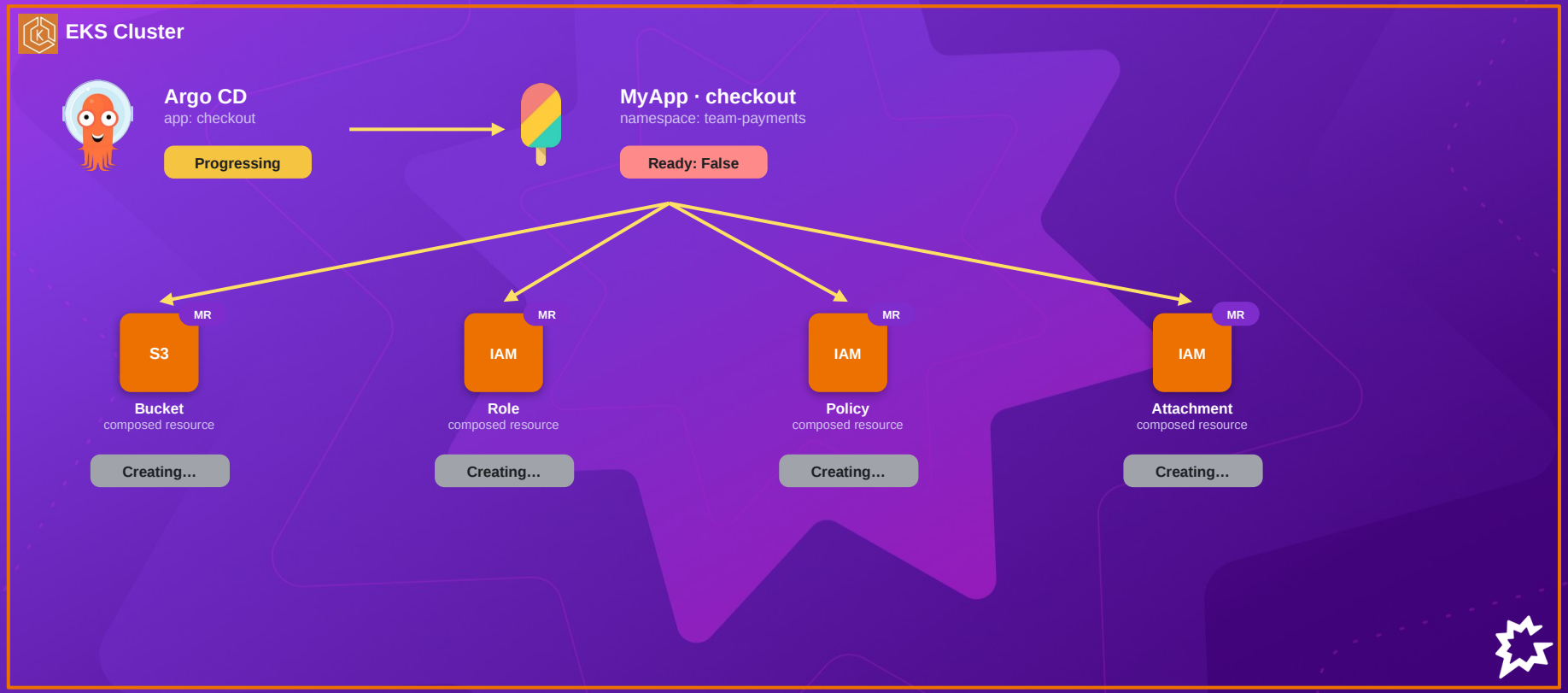
1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.



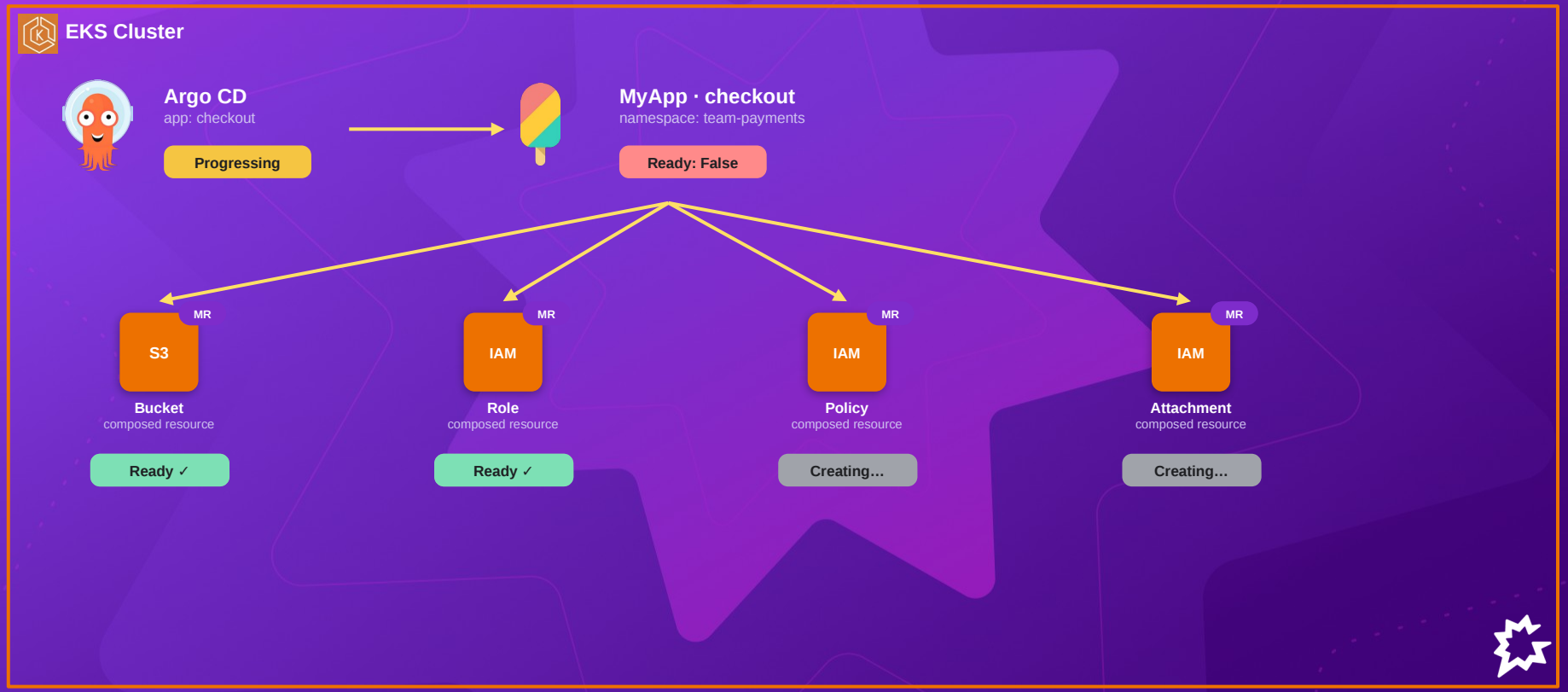
Challenge 3 · Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.



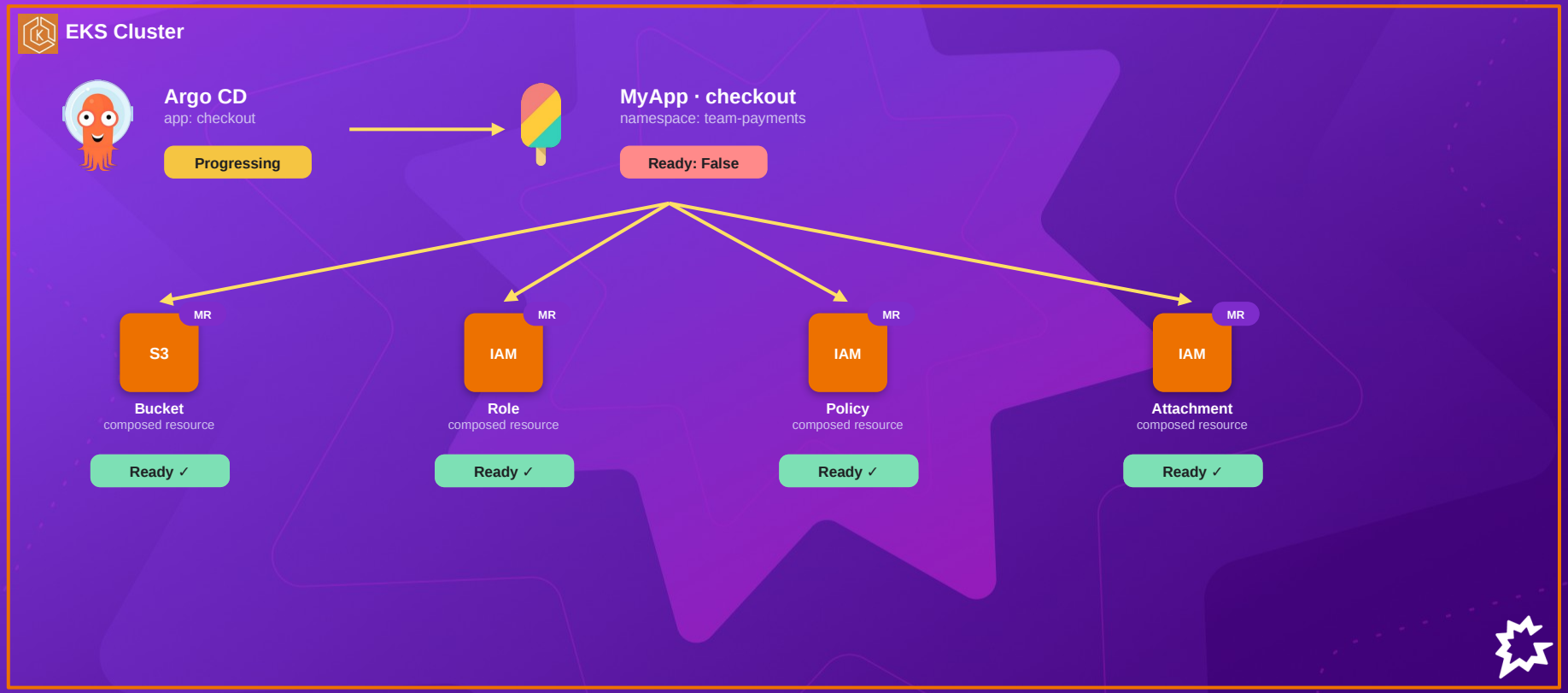
Challenge 3 · Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.



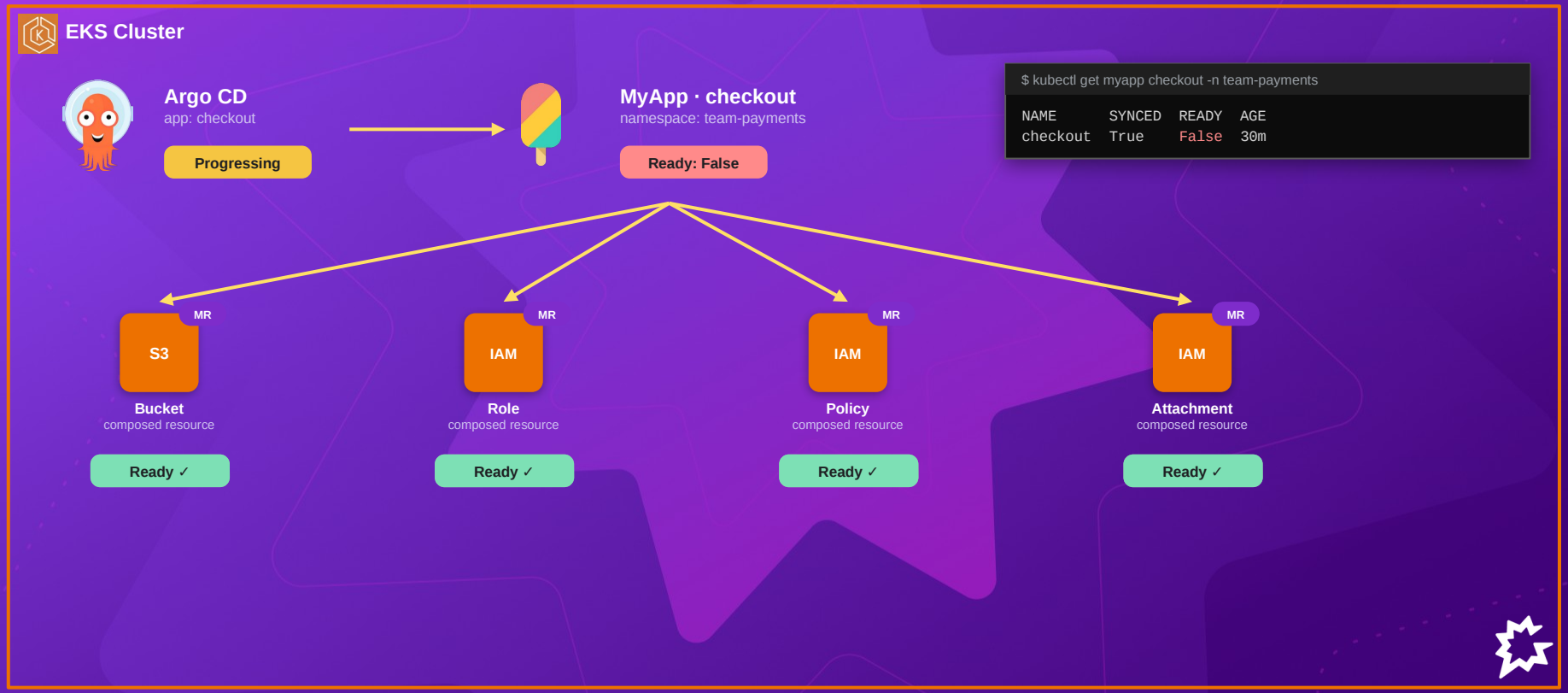
Challenge 3 · Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.



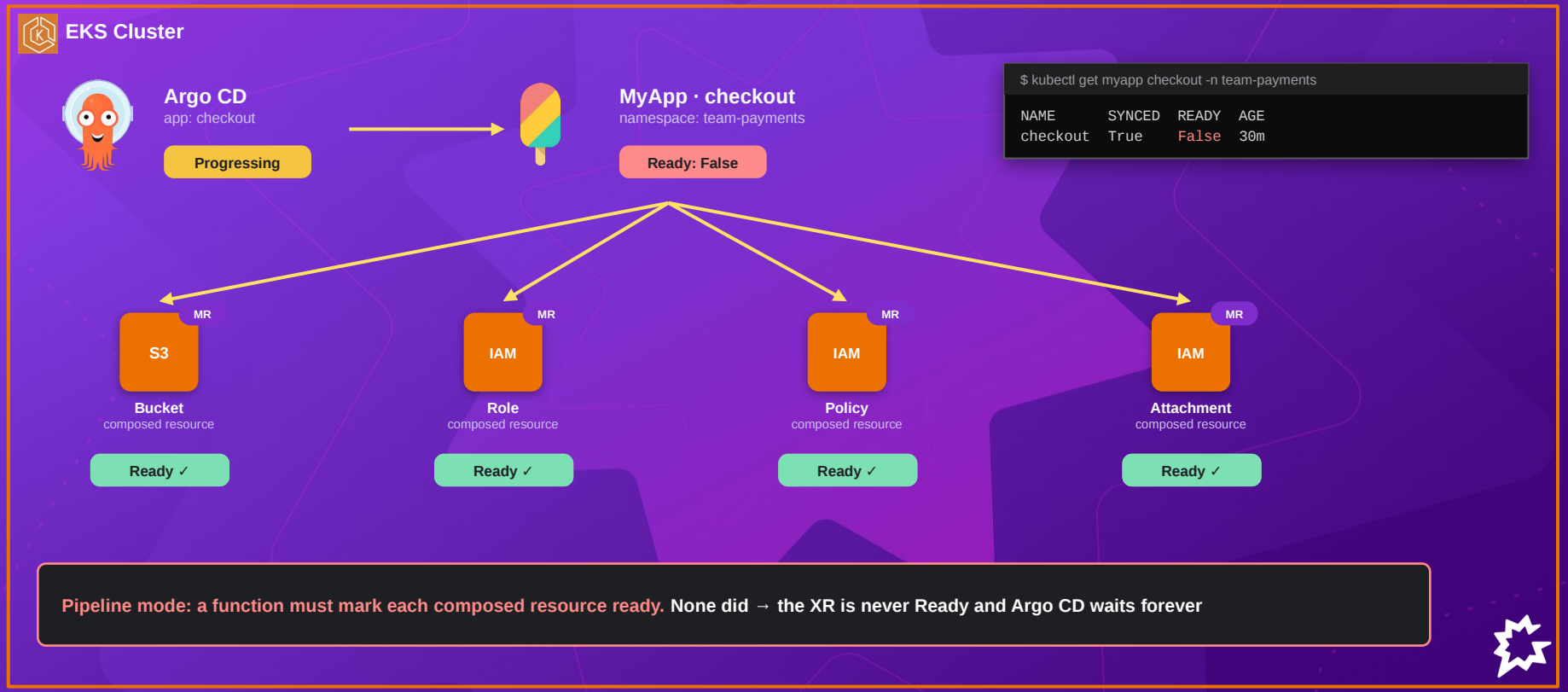
Challenge 3 · Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.



Challenge 3 · Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

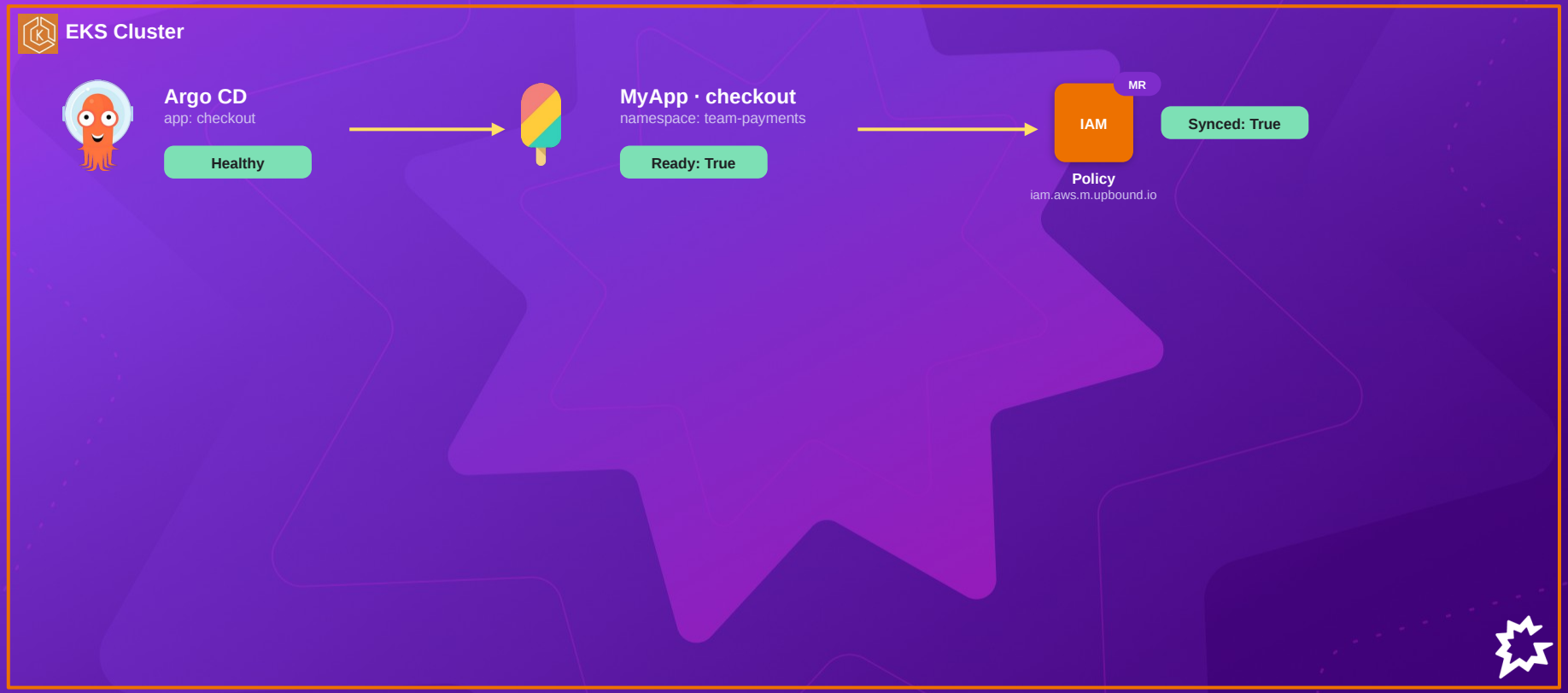
1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.



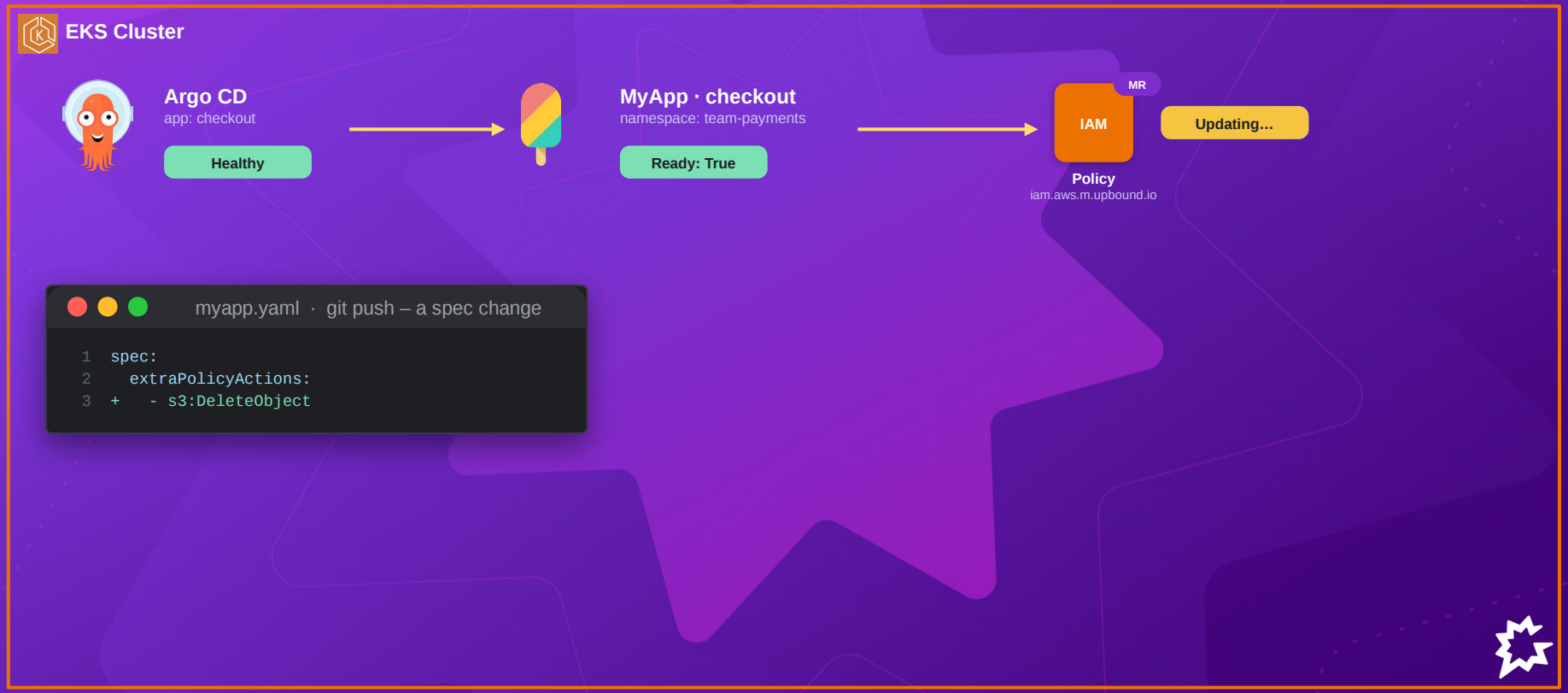
Challenge 4 · Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.



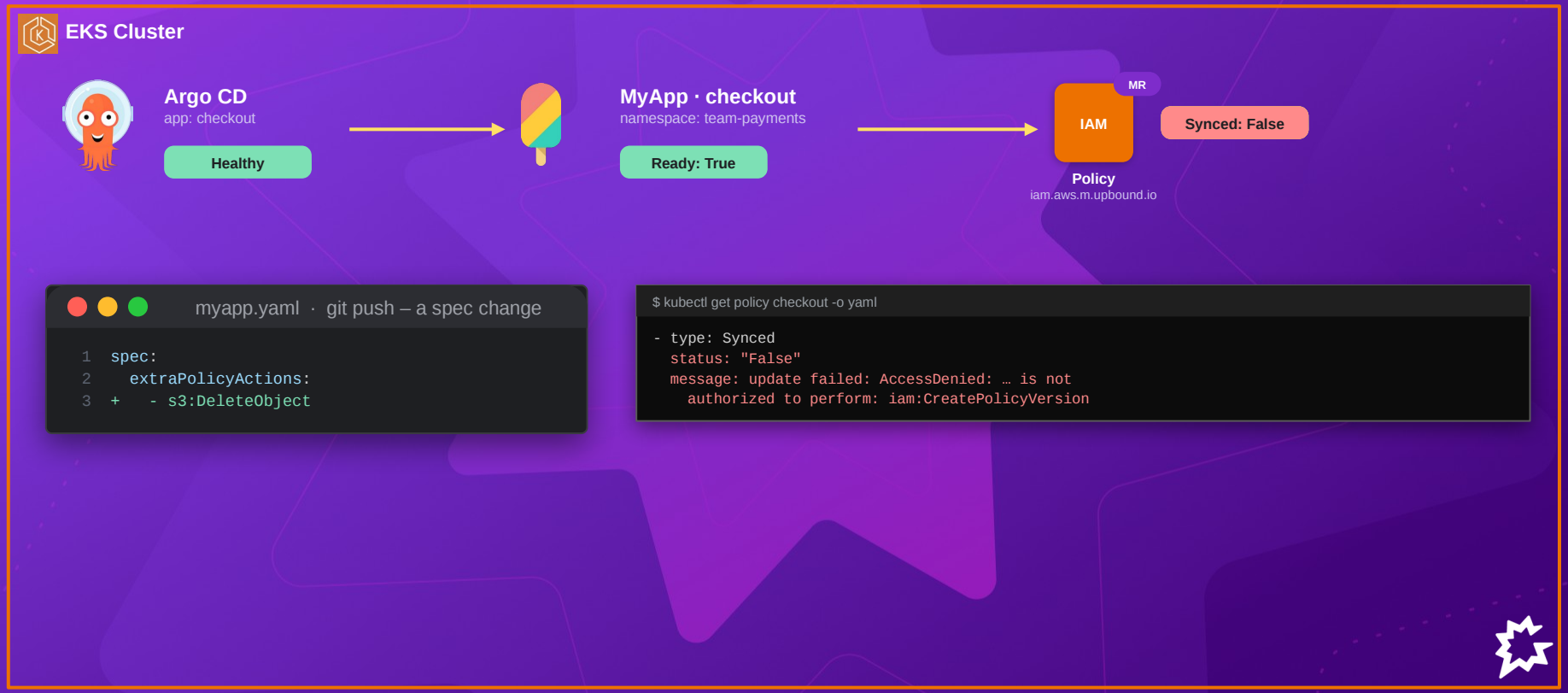
Challenge 4 · Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.



Challenge 4 · Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.



myapp.yaml · git push – a spec change

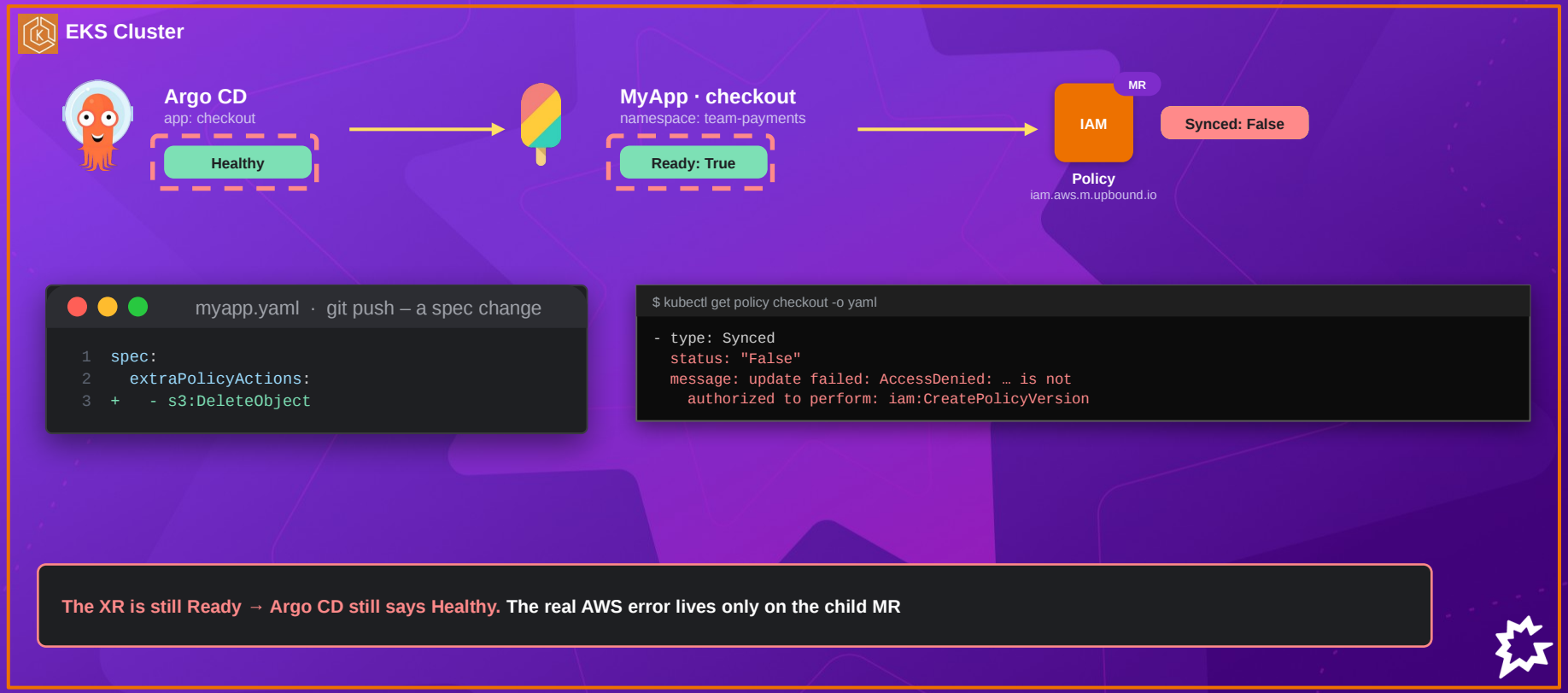
```
1  spec:
2    extraPolicyActions:
3  +   - s3:DeleteObject
```

$ kubectl get policy checkout -o yaml

```
- type: Synced
  status: "False"
  message: update failed: AccessDenied: … is not
    authorized to perform: iam:CreatePolicyVersion
```

Challenge 4 · Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.



myapp.yaml · git push – a spec change

```
1  spec:
2    extraPolicyActions:
3  +   - s3:DeleteObject
```

$ kubectl get policy checkout -o yaml

```
- type: Synced
  status: "False"
  message: update failed: AccessDenied: … is not
    authorized to perform: iam:CreatePolicyVersion
```

The XR is still Ready → Argo CD still says Healthy. The real AWS error lives only on the child MR

Meet MyApp: our running example

One developer request → four functions → five AWS resources



Meet MyApp: our running example

One developer request → four functions → five AWS resources

myapp.yaml · what the developer writes

```yaml
apiVersion: crossplane.best-
  practice.com/v1alpha1
kind: MyApp
metadata:
  name: checkout
  namespace: team-payments
spec:
  team: payments
  region: us-east-1
```

Meet MyApp: our running example

One developer request → four functions → five AWS resources

myapp.yaml · what the developer writes

```yaml
apiVersion: crossplane.best-
  practice.com/v1alpha1
kind: MyApp
metadata:
  name: checkout
  namespace: team-payments
spec:
  team: payments
  region: us-east-1
```

COMPOSITION PIPELINE · RUNS ON EVERY RECONCILE

1 **render**
function-go-templating · in-cluster render with observed state

Meet MyApp: our running example

One developer request → four functions → five AWS resources

myapp.yaml · what the developer writes

```yaml
apiVersion: crossplane.best-
  practice.com/v1alpha1
kind: MyApp
metadata:
  name: checkout
  namespace: team-payments
spec:
  team: payments
  region: us-east-1
```

COMPOSITION PIPELINE · RUNS ON EVERY RECONCILE

1 **render**
function-go-templating · in-cluster render with observed state

2 **auto-ready**
function-auto-ready · child Ready → XR Ready



Meet MyApp: our running example

One developer request → four functions → five AWS resources

myapp.yaml · what the developer writes

```yaml
apiVersion: crossplane.best-
  practice.com/v1alpha1
kind: MyApp
metadata:
  name: checkout
  namespace: team-payments
spec:
  team: payments
  region: us-east-1
```

COMPOSITION PIPELINE · RUNS ON EVERY RECONCILE

1. **render**
   function-go-templating · in-cluster render with observed state
2. **auto-ready**
   function-auto-ready · child Ready → XR Ready
3. **sequence**
   function-sequencer · hold resources until deps are Ready

Meet MyApp: our running example

One developer request → four functions → five AWS resources

myapp.yaml · what the developer writes

```yaml
apiVersion: crossplane.best-
  practice.com/v1alpha1
kind: MyApp
metadata:
  name: checkout
  namespace: team-payments
spec:
  team: payments
  region: us-east-1
```

COMPOSITION PIPELINE · RUNS ON EVERY RECONCILE

1. **render**
   function-go-templating · in-cluster render with observed state
2. **auto-ready**
   function-auto-ready · child Ready → XR Ready
3. **sequence**
   function-sequencer · hold resources until deps are Ready
4. **status**
   function-status-transformer · lift child errors onto the XR

Meet MyApp: our running example

One developer request → four functions → five AWS resources

myapp.yaml · what the developer writes

```yaml
apiVersion: crossplane.best-
  practice.com/v1alpha1
kind: MyApp
metadata:
  name: checkout
  namespace: team-payments
spec:
  team: payments
  region: us-east-1
```

COMPOSITION PIPELINE · RUNS ON EVERY RECONCILE

1. **render**
   function-go-templating · in-cluster render with observed state
2. **auto-ready**
   function-auto-ready · child Ready → XR Ready
3. **sequence**
   function-sequencer · hold resources until deps are Ready
4. **status**
   function-status-transformer · lift child errors onto the XR

WHAT IT COMPOSES



Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

what Crossplane assumes

```yaml
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
# legacy cluster-scoped MR (iam.aws.upbound.io)
providerConfigRef: {name: default}
```

what the platform installs

```yaml
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```



Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

what Crossplane assumes

```yaml
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
```

what the platform installs

```yaml
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```

Omit it

The MR's CRD fills it in: v2 namespaced MRs get kind: ClusterProviderConfig, name: default.

Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

what Crossplane assumes

```yaml
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
# legacy cluster-scoped MR (iam.aws.upbound.io)
providerConfigRef: {name: default}
```

what the platform installs

```yaml
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```

One default per cluster

Same name on every cluster – each cluster's default points at its own AWS account.

Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

what Crossplane assumes

```yaml
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
# legacy cluster-scoped MR (iam.aws.upbound.io)
providerConfigRef: {name: default}
```

what the platform installs

```yaml
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```

The account lives here

IRSA / Pod Identity on the cluster – the Composition never knows which account it runs in.

Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

what Crossplane assumes

```yaml
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
# legacy cluster-scoped MR (iam.aws.upbound.io)
providerConfigRef: {name: default}
```

what the platform installs

```yaml
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```

1 **Less boilerplate**
Developers never write it: the XRD defaults providerConfigRef to ClusterProviderConfig / default and the Composition passes it on.



Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

what Crossplane assumes

```yaml
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
# legacy cluster-scoped MR (iam.aws.upbound.io)
providerConfigRef: {name: default}
```

what the platform installs

```yaml
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```

1. **Less boilerplate**
   Developers never write it: the XRD defaults providerConfigRef to ClusterProviderConfig / default and the Composition passes it on.

2. **Portable**
   Same Composition on every cluster – each cluster's default points at its own account.



Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

what Crossplane assumes

```yaml
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
# legacy cluster-scoped MR (iam.aws.upbound.io)
providerConfigRef: {name: default}
```

what the platform installs

```yaml
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```

1. **Less boilerplate**
   Developers never write it: the XRD defaults providerConfigRef to ClusterProviderConfig / default and the Composition passes it on.

2. **Portable**
   Same Composition on every cluster – each cluster's default points at its own account.

3. **No 'not found' typos**
   You can't misspell a name you never write. Imported, observed and rendered MRs just work.

Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

what Crossplane assumes

```yaml
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
# legacy cluster-scoped MR (iam.aws.upbound.io)
providerConfigRef: {name: default}
```

what the platform installs

```yaml
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```

1. **Less boilerplate**
   Developers never write it: the XRD defaults providerConfigRef to ClusterProviderConfig / default and the Composition passes it on.

2. **Portable**
   Same Composition on every cluster – each cluster's default points at its own account.

3. **No 'not found' typos**
   You can't misspell a name you never write. Imported, observed and rendered MRs just work.

Name it explicitly only when you need a second one: cross-account, or a namespaced ProviderConfig per team.

In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	⎈ helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster

In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	⎈ helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster
Inputs	✗ values.yaml	✓ the live XR + observed state of every child

In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	⎈ helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster
Inputs	✗ values.yaml	✓ the live XR + observed state of every child
Cloud IDs	✗ unknown – copy ARNs into values	✓ .observed.resources → ARNs, IDs, status



In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	⎈ helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster
Inputs	✗ values.yaml	✓ the live XR + observed state of every child
Cloud IDs	✗ unknown – copy ARNs into values	✓ .observed.resources → ARNs, IDs, status
Drift	✗ only on the next sync	✓ continuous – the loop never stops



In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	⎈ helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster
Inputs	✗ values.yaml	✓ the live XR + observed state of every child
Cloud IDs	✗ unknown – copy ARNs into values	✓ .observed.resources → ARNs, IDs, status
Drift	✗ only on the next sync	✓ continuous – the loop never stops

composition.yaml · step 1 render

```
1  delims: {left: '[[', right: ']]'}
2  [[- $observed := .observed.resources ]]
3  [[- $bucketArn := dig "bucket" "resource"
4        "status" "atProvider" "arn" "" $observed ]]
5  [[- $logsArn := dig "shared-logs" ...
6        "manifest" "status" "atProvider" "arn" ]]
7
8  [[- if and $bucketArn $logsArn ]]
9  apiVersion: iam.aws.m.upbound.io/v1beta1
10 kind: Policy
11 spec:
12   forProvider:
13     policy: [[ ... $bucketArn $logsArn ... ]]
14 [[- end ]]
15
16 # Helm only parses {{ }} - it ships [[ ]] as-is
17 # so Crossplane renders it, in-cluster
```

In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	✱ helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster
Inputs	✗ values.yaml	✓ the live XR + observed state of every child
Cloud IDs	✗ unknown – copy ARNs into values	✓ .observed.resources → ARNs, IDs, status
Drift	✗ only on the next sync	✓ continuous – the loop never stops

Read what AWS reported

dig pulls the bucket ARN out of .observed.resources – empty until AWS has created it.

composition.yaml · step 1 render

```
1  delims: {left: '[[', right: ']]'}
2  [[- $observed := .observed.resources ]]
3  [[- $bucketArn := dig "bucket" "resource"
4        "status" "atProvider" "arn" "" $observed ]]
5  [[- $logsArn := dig "logs" "resource"
6        "manifest" "status" "atProvider" "arn" ]]
7
8  [[- if and $bucketArn $logsArn ]]
9  apiVersion: iam.aws.m.upbound.io/v1beta1
10 kind: Policy
11 spec:
12   forProvider:
13     policy: [[ ... $bucketArn $logsArn ... ]]
14 [[- end ]]
15
16 # Helm only parses {{ }} - it ships [[ ]] as-is
17 # so Crossplane renders it, in-cluster
```

In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	✱ helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster
Inputs	✗ values.yaml	✓ the live XR + observed state of every ch
Cloud IDs	✗ unknown – copy ARNs into values	✓ .observed.resources → ARNs, IDs, status
Drift	✗ only on the next sync	✓ continuous – the loop never stops

Not known yet? Don't emit it

The Policy is rendered only once both ARNs exist. The next reconcile will emit it.

composition.yaml · step 1 render

```
1  delims: {left: '[[', right: ']]'}
2  [[- $observed := .observed.resources ]]
3  [[- $bucketArn := dig "bucket" "resource"
4        "status" "atProvider" "arn" "" $observed ]]
5  [[- $logsArn := dig "shared-logs" ...
6        "manifest" "status" "atProvider" "arn" ]]
7
8  [[- if and $bucketArn $logsArn ]]
9  apiVersion: iam.aws.m.upbound.io/v1beta1
10 kind: Policy
11 spec:
12   forProvider:
13     policy: [[ ... $bucketArn $logsArn ... ]]
14 [[- end ]]
15
16 # Helm only parses {{ }} - it ships [[ ]] as-is
17 # so Crossplane renders it, in-cluster
```

In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	⎈ helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster
Inputs	✗ values.yaml	✓ the live XR + observed state of every child
Cloud IDs	✗ unknown – copy ARNs into values	✓ .observed.resources → ARNs, IDs, status
Drift	✗ only on the next sync	✓ continuous – the loop never stops

composition.yaml · step 1 render

```
1  delims: {left: '[[', right: ']]'}
2  [[- $observed := .observed.resources ]]
3  [[- $bucketArn := dig "bucket" "resource"
4        "status" "atProvider" "arn" "" $observed ]]
5  [[- $logsArn := dig "shared-logs" ...
6        "manifest" "status" "atProvider" "arn" ]]
7
8  [[- if and $bucketArn $logsArn ]]
9  apiVersion: iam.aws.m.upbound.io/v1beta1
10 kind: Policy
11 spec:
12   forProvider:
13     policy: [[ ... $bucketArn $logsArn ... ]]
14 [[- end ]]
15
16 # Helm only parses {{ }} - it ships [[ ]] as-is
17 # so Crossplane renders it, in-cluster
```

Helm ships the template, Crossplane renders it. **Not known yet? Don't emit it – the next reconcile will.**

Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

object.yaml · composed resource, read-only window

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  forProvider:
    manifest:          # the thing to watch
      apiVersion: s3.aws.m.upbound.io/v1beta1
      kind: Bucket
      metadata:
        name: platform-logs
        namespace: platform
```



Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

object.yaml · composed resource, read-only window

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  forProvider:
    manifest:          # the thing to watch
      apiVersion: s3.aws.m.upbound.io/v1beta1
      kind: Bucket
      metadata:
        name: platform-logs
        namespace: platform
```

Read-only

The provider only GETs the bucket – it never creates, updates or deletes it.

Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

object.yaml · composed resource, read-only window

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  manifest:          # the thing to watch
    apiVersion: s3.aws.m.upbound.io/v1beta1
    kind: Bucket
    metadata:
      name: platform-logs
      namespace: platform
```

Readiness from the object

The default waits for a create that never happens on Observe – the XR would never be Ready.



Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

object.yaml · composed resource, read-only window

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  forProvider:
    manifest:          # the thing to watch
      apiVersion: s3.aws.m.upbound.io/v1beta1
      kind: Bucket
      metadata:
        name: platform-logs
        namespace: platform
```

The thing to watch

Any existing object – its whole state lands in status.atProvider.manifest.

Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

object.yaml · composed resource, read-only window

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  forProvider:
    manifest:          # the thing to watch
      apiVersion: s3.aws.m.upbound.io/v1beta1
      kind: Bucket
      metadata:
        name: platform-logs
        namespace: platform
```

1 **Observe**
Provider GETs the bucket. Never creates, updates or deletes it – it belongs to the platform team.



Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

object.yaml · composed resource, read-only window

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  forProvider:
    manifest:          # the thing to watch
      apiVersion: s3.aws.m.upbound.io/v1beta1
      kind: Bucket
      metadata:
        name: platform-logs
        namespace: platform
```

1 **Observe**
Provider GETs the bucket. Never creates, updates or deletes it – it belongs to the platform team.

2 **Read status**
Whole object lands in status.atProvider.manifest → …manifest.status.atProvider.arn

Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

object.yaml · composed resource, read-only window

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  forProvider:
    manifest:          # the thing to watch
      apiVersion: s3.aws.m.upbound.io/v1beta1
      kind: Bucket
      metadata:
        name: platform-logs
        namespace: platform
```

1 **Observe**
Provider GETs the bucket. Never creates, updates or deletes it – it belongs to the platform team.

2 **Read status**
Whole object lands in status.atProvider.manifest → …manifest.status.atProvider.arn

3 **Use it**
go-templating reads it from .observed.resources and writes it into the app's IAM Policy.

Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

object.yaml · composed resource, read-only window

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  forProvider:
    manifest:          # the thing to watch
      apiVersion: s3.aws.m.upbound.io/v1beta1
      kind: Bucket
      metadata:
        name: platform-logs
        namespace: platform
```

1 **Observe**
Provider GETs the bucket. Never creates, updates or deletes it – it belongs to the platform team.

2 **Read status**
Whole object lands in status.atProvider.manifest → …manifest.status.atProvider.arn

3 **Use it**
go-templating reads it from .observed.resources and writes it into the app's IAM Policy.

Gotchas: readiness DeriveFromObject (SuccessfulCreate never fires on Observe) · RBAC needs patch – SSA dry-run runs even for Observe

Dependencies – function-sequencer

Hold a composed resource back until the ones it depends on are Ready

composition.yaml · step 3

```yaml
- step: sequence
  functionRef:
    name: function-sequencer
  input:
    apiVersion: sequencer.fn.crossplane.io/v1beta1
    kind: Input
    rules:
    - sequence:
      - role
      - policy
      - role-policy-attachment
```



Dependencies – function-sequencer

Hold a composed resource back until the ones it depends on are Ready

composition.yaml · step 3

```yaml
- step: sequence
  functionRef:
    name: function-sequencer
  input:
    apiVersion: sequencer.fn.crossplane.io/v1beta1
    kind: Input
    rules:
      - sequence:
        - role
        - policy
        - role-policy-attachment
```

One rule, in order

Each resource waits until the ones before it are Ready – until then it is left out of the desired state.

Dependencies – function-sequencer

Hold a composed resource back until the ones it depends on are Ready

composition.yaml · step 3

```yaml
- step: sequence
  functionRef:
    name: function-sequencer
  input:
    apiVersion: sequencer.fn.crossplane.io/v1beta1
    kind: Input
    rules:
    - sequence:
      - role
      - policy
      - role-policy-attachment
```

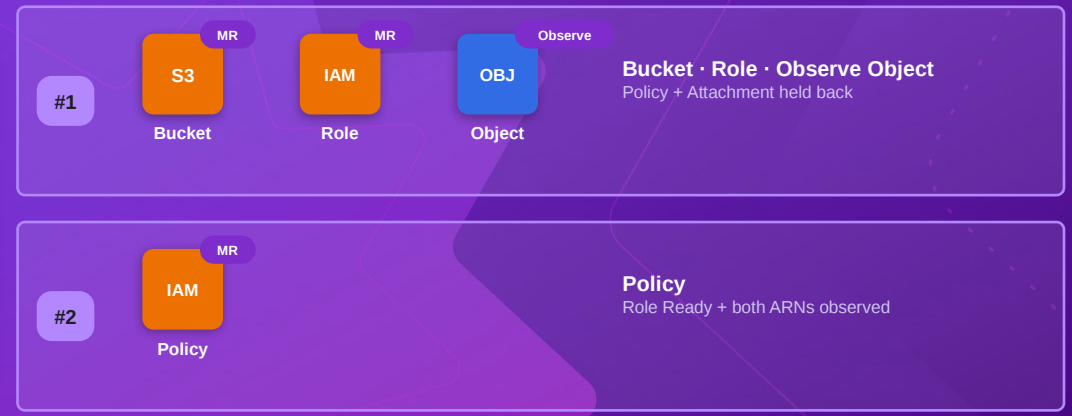


Dependencies – function-sequencer

Hold a composed resource back until the ones it depends on are Ready

composition.yaml · step 3

```yaml
- step: sequence
  functionRef:
    name: function-sequencer
  input:
    apiVersion: sequencer.fn.crossplane.io/v1beta1
    kind: Input
    rules:
    - sequence:
      - role
      - policy
      - role-policy-attachment
```

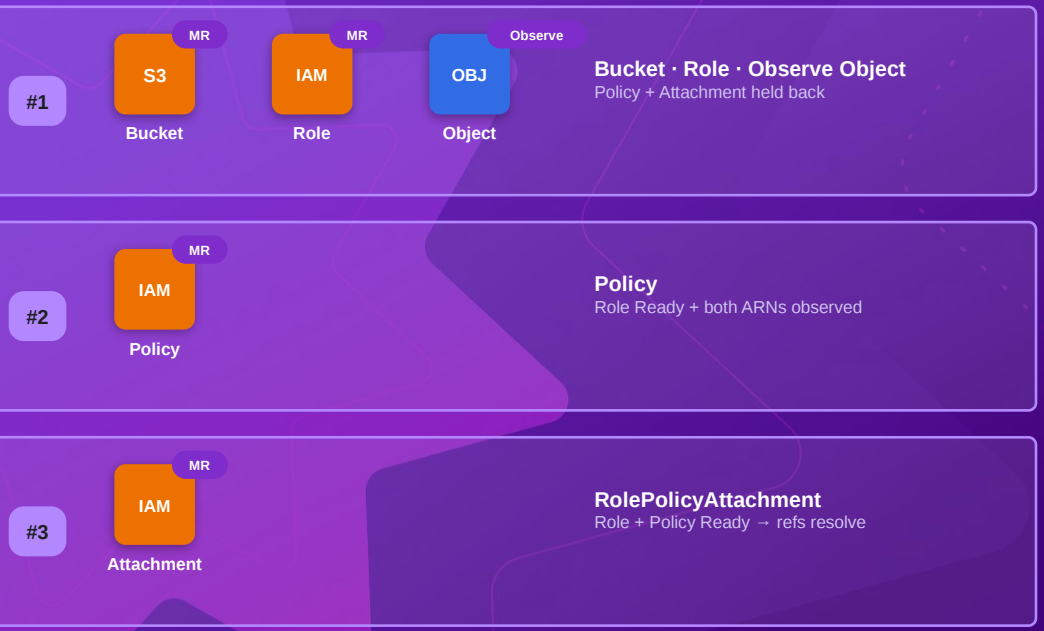


Dependencies – function-sequencer

Hold a composed resource back until the ones it depends on are Ready

composition.yaml · step 3

```yaml
- step: sequence
  functionRef:
    name: function-sequencer
  input:
    apiVersion: sequencer.fn.crossplane.io/v1beta1
    kind: Input
    rules:
    - sequence:
      - role
      - policy
      - role-policy-attachment
```



Dependencies – function-sequencer

Hold a composed resource back until the ones it depends on are Ready

composition.yaml · step 3

```yaml
- step: sequence
  functionRef:
    name: function-sequencer
  input:
    apiVersion: sequencer.fn.crossplane.io/v1beta1
    kind: Input
    rules:
    - sequence:
      - role
      - policy
      - role-policy-attachment
```

$ kubectl events --for myapp/checkout -n team-payments

```
Delaying creation of resource(s) matching
"role-policy-attachment" because "policy"
is not fully ready (0 of 1)
```



Dependencies – function-sequencer

Hold a composed resource back until the ones it depends on are Ready

composition.yaml · step 3

```yaml
- step: sequence
  functionRef:
    name: function-sequencer
  input:
    apiVersion: sequencer.fn.crossplane.io/v1beta1
    kind: Input
    rules:
    - sequence:
      - role
      - policy
      - role-policy-attachment
```

$ kubectl events --for myapp/checkout -n team-payments

```
Delaying creation of resource(s) matching
"role-policy-attachment" because "policy"
is not fully ready (0 of 1)
```



No 'referenced field was empty' error storm. Deletes too: enableDeletionSequencing creates Usages → reverse order

Ready – function-auto-ready

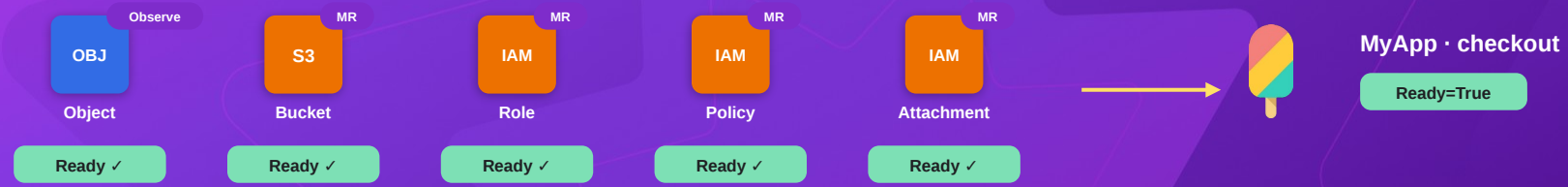
The XR is Ready only when every composed resource reports Ready=True



Ready – function-auto-ready

The XR is Ready only when every composed resource reports Ready=True

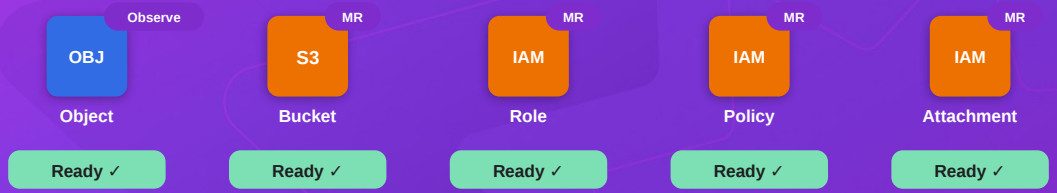
COMPOSED RESOURCES → ONE ANSWER



Ready – function-auto-ready

The XR is Ready only when every composed resource reports Ready=True

COMPOSED RESOURCES → ONE ANSWER





MyApp · checkout

Ready=True

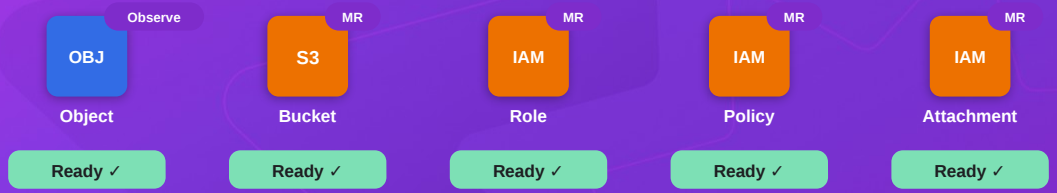
Why a function?

Composition functions don't mark anything Ready on their own. Without auto-ready (or a manual gotemplating…/ready annotation) the XR stays Ready=False forever.

Ready – function-auto-ready

The XR is Ready only when every composed resource reports Ready=True

COMPOSED RESOURCES → ONE ANSWER





MyApp · checkout

Ready=True

Why a function?

Composition functions don't mark anything Ready on their own. Without auto-ready (or a manual gotemplating…/ready annotation) the XR stays Ready=False forever.

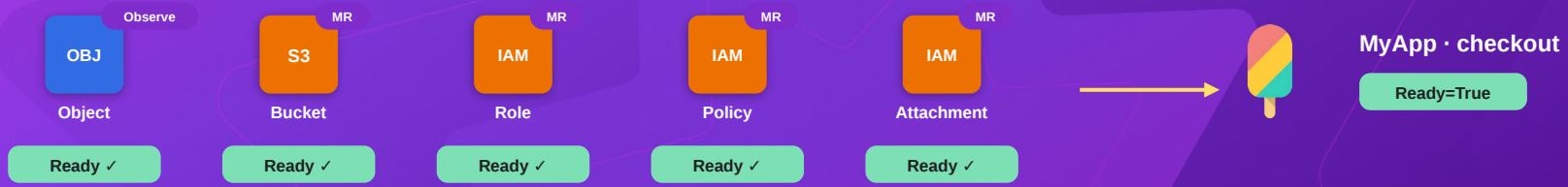
Where in the pipeline

Right after render, before the sequencer – the sequencer decides what to create from the readiness auto-ready just computed.

Ready – function-auto-ready

The XR is Ready only when every composed resource reports Ready=True

COMPOSED RESOURCES → ONE ANSWER



Why a function?

Composition functions don't mark anything Ready on their own. Without auto-ready (or a manual gotemplating…/ready annotation) the XR stays Ready=False forever.

Where in the pipeline

Right after render, before the sequencer – the sequencer decides what to create from the readiness auto-ready just computed.

Ready ≠ Healthy

If an UPDATE fails, the old AWS resource still exists, so the child stays Ready=True – and so does the XR. ArgoCD says Healthy.

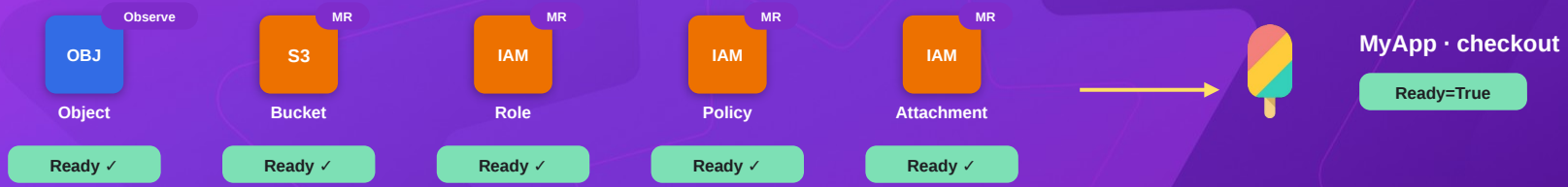
```
$ kubectl -n team-payments get myapp checkout

NAME       HEALTHY   ROLE-ARN                                          SYNCED   READY
checkout   False     arn:aws:iam::123456789012:role/team-payments-checkout   True     True
```

Ready – function-auto-ready

The XR is Ready only when every composed resource reports Ready=True

COMPOSED RESOURCES → ONE ANSWER



Why a function?

Composition functions don't mark anything Ready on their own. Without auto-ready (or a manual gotemplating…/ready annotation) the XR stays Ready=False forever.

Where in the pipeline

Right after render, before the sequencer – the sequencer decides what to create from the readiness auto-ready just computed.

Ready ≠ Healthy

If an UPDATE fails, the old AWS resource still exists, so the child stays Ready=True – and so does the XR. ArgoCD says Healthy.

```
$ kubectl -n team-payments get myapp checkout

NAME       HEALTHY   ROLE-ARN                                          SYNCED   READY
checkout   False     arn:aws:iam::123456789012:role/team-payments-checkout   True     True
```

Ready tells you it exists – not that your last change worked. That's the next function's job.

ArgoCD health – function-status-transformer

Lift the child's real AWS error onto the XR, so ArgoCD can turn it Degraded

composition.yaml · step 4

```yaml
- step: status
  functionRef: {name: function-status-transformer}
  input:
    kind: StatusTransformation
    statusConditionHooks:
    - matchers:
      - type: AnyResourceMatchesAnyCondition
        resources: [{name: ".*"}]
        conditions:
        - {type: Synced, status: "False",
           reason: ReconcileError,
           message: "(?P<Error>(?s).+)"}
      setConditions:
      - target: Composite
        condition: {type: ResourcesHealthy,
          status: "False", message: "{{ .Error }}"}
```

ArgoCD health – function-status-transformer

Lift the child's real AWS error onto the XR, so ArgoCD can turn it Degraded

composition.yaml · step 4

```yaml
- step: status
  functionRef: {name: function-status-transformer}
  input:
    kind: StatusTransformation
    statusConditionHooks:
    - matchers:
      - type: AnyResourceMatchesAnyCondition
        resources: [{name: ".*"}]
        conditions:
        - {type: Synced, status: "False",
           reason: ReconcileError,
           message: "(?P<Error>(?s).+)"}
      condition: {type: ResourcesHealthy,
        status: "False", message: "{{ .Error }}"}
```

Match any failing child

Any composed resource with Synced=False / ReconcileError – the regex captures its message.



ArgoCD health – function-status-transformer

Lift the child's real AWS error onto the XR, so ArgoCD can turn it Degraded

composition.yaml · step 4

```yaml
- step: status
  functionRef: {name: function-status-transformer}
  input:
    kind: StatusTransformation
    statusConditionHooks:
    - matchers:
      - type: AnyResourceMatchesAnyCondition
        resources: [{name: ".*"}]
        conditions:
        - {type: Synced, status: "False",
           reason: ReconcileError,
           message: "(?P<Error>(?s).+)"}
setConditions:
- target: Composite
  condition: {type: ResourcesHealthy,
    status: "False", message: "{{ .Error }}"}
```

Set it on the XR

ResourcesHealthy=False on the MyApp, carrying the child's real AWS message.



ArgoCD health – function-status-transformer

Lift the child's real AWS error onto the XR, so ArgoCD can turn it Degraded

composition.yaml · step 4

```yaml
- step: status
  functionRef: {name: function-status-transformer}
  input:
    kind: StatusTransformation
    statusConditionHooks:
    - matchers:
      - type: AnyResourceMatchesAnyCondition
        resources: [{name: ".*"}]
        conditions:
        - {type: Synced, status: "False",
           reason: ReconcileError,
           message: "(?P<Error>(?s).+)"}
      setConditions:
      - target: Composite
        condition: {type: ResourcesHealthy,
          status: "False", message: "{{ .Error }}"}
```

argocd-cm · Lua health check

```
# argocd-cm
resource.customizations.health.
  crossplane.best-practice.com_MyApp: |
  if c.type == "ResourcesHealthy" and
     c.status == "False" then
    return {status="Degraded", message=c.message}
```



ArgoCD health – function-status-transformer

Lift the child's real AWS error onto the XR, so ArgoCD can turn it Degraded

composition.yaml · step 4

```yaml
- step: status
  functionRef: {name: function-status-transformer}
  input:
    kind: StatusTransformation
    statusConditionHooks:
    - matchers:
      - type: AnyResourceMatchesAnyCondition
        resources: [{name: ".*"}]
        conditions:
        - {type: Synced, status: "False",
           reason: ReconcileError,
           message: "(?P<Error>(?s).+)"}
      setConditions:
      - target: Composite
        condition: {type: ResourcesHealthy,
          status: "False", message: "{{ .Error }}"}
```

argocd-cm · Lua health check

```
# argocd-cm
resource.customizations.health.
  crossplane.best-practice.com_MyApp: |
  if c.type == "ResourcesHealthy" and
     c.status == "False" then
    return {status="Degraded", message=c.message}
```



Without

♥ Healthy

error hidden on a child Policy

ArgoCD health – function-status-transformer

Lift the child's real AWS error onto the XR, so ArgoCD can turn it Degraded

composition.yaml · step 4

```yaml
- step: status
  functionRef: {name: function-status-transformer}
  input:
    kind: StatusTransformation
    statusConditionHooks:
    - matchers:
      - type: AnyResourceMatchesAnyCondition
        resources: [{name: ".*"}]
        conditions:
        - {type: Synced, status: "False",
           reason: ReconcileError,
           message: "(?P<Error>(?s).+)"}
      setConditions:
      - target: Composite
        condition: {type: ResourcesHealthy,
          status: "False", message: "{{ .Error }}"}
```

argocd-cm · Lua health check

```
# argocd-cm
resource.customizations.health.
  crossplane.best-practice.com_MyApp: |
  if c.type == "ResourcesHealthy" and
     c.status == "False" then
    return {status="Degraded", message=c.message}
```



Without

♥ Healthy

error hidden on a child Policy



With

✗ Degraded

MalformedPolicyDocument: Actions must be prefaced by a vendor…

$ kubectl get myapp checkout -n team-payments -o yaml

```
ResourcesHealthy=False  ReconcileError  update failed: … CreatePolicyVersion … 400 …
MalformedPolicyDocument: Actions/Conditions must be prefaced by a vendor, e.g., iam, sdb, ec2
```

ArgoCD health – function-status-transformer

Lift the child's real AWS error onto the XR, so ArgoCD can turn it Degraded

composition.yaml · step 4

```yaml
- step: status
  functionRef: {name: function-status-transformer}
  input:
    kind: StatusTransformation
    statusConditionHooks:
    - matchers:
      - type: AnyResourceMatchesAnyCondition
        resources: [{name: ".*"}]
        conditions:
        - {type: Synced, status: "False",
           reason: ReconcileError,
           message: "(?P<Error>(?s).+)"}
      setConditions:
      - target: Composite
        condition: {type: ResourcesHealthy,
          status: "False", message: "{{ .Error }}"}
```

argocd-cm · Lua health check

```lua
# argocd-cm
resource.customizations.health.
  crossplane.best-practice.com_MyApp: |
  if c.type == "ResourcesHealthy" and
     c.status == "False" then
    return {status="Degraded", message=c.message}
```



Without

♥ Healthy

error hidden on a child Policy



With

✗ Degraded

MalformedPolicyDocument: Actions must be prefaced by a vendor…

$ kubectl get myapp checkout -n team-payments -o yaml

```
ResourcesHealthy=False  ReconcileError  update failed: … CreatePolicyVersion … 400 …
MalformedPolicyDocument: Actions/Conditions must be prefaced by a vendor, e.g., iam, sdb, ec2
```

The developer sees the AWS message on the object they deployed – no kubectl describe hunt across 5 children.

Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

xr.yaml · the XR to render

```yaml
# helm template checkout charts/myapp-claim -n team-payments
apiVersion: crossplane.best-practice.com/v1alpha1
kind: MyApp
metadata:
  name: checkout
  namespace: team-payments
spec:
  team: payments
  region: us-east-1
```



Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

1 **xr.yaml**
the XR to render

composition.yaml · the pipeline to run

```yaml
# helm template charts/myapp-composition --show-only templates/composition.yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef: {apiVersion: crossplane.best-practice.com/v1alpha1, kind: MyApp}
  mode: Pipeline
  pipeline:
  - step: render       # function-go-templating
  - step: auto-ready   # function-auto-ready
  - step: sequence     # function-sequencer
  - step: status       # function-status-transformer
```



Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

1. **xr.yaml**
   the XR to render

2. **composition.yaml**
   the pipeline to run

functions.yaml · which function images

```yaml
# crossplane/functions.yaml – the same file the cluster applies
apiVersion: pkg.crossplane.io/v1
kind: Function
metadata:
  name: function-go-templating
spec:
  package: xpkg.crossplane.io/crossplane-contrib/function-go-templating:v0.13.0
---
# ... + function-auto-ready, function-sequencer, function-status-transformer
# render pulls each one and runs it in Docker
```



Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

1. **xr.yaml**
   the XR to render
2. **composition.yaml**
   the pipeline to run
3. **functions.yaml**
   which function images

xrd.yaml · schema + defaults

```yaml
# helm template charts/myapp-composition --show-only templates/xrd.yaml
apiVersion: apiextensions.crossplane.io/v2
kind: CompositeResourceDefinition
spec:
  scope: Namespaced
  ...
    region:            {default: us-east-1}
    sharedLogsBucket:  {default: {name: platform-logs, namespace: platform}}
    providerConfigRef: {kind: ClusterProviderConfig, name: default}
# --xrd applies these defaults, exactly like the API server
```



Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

1. **xr.yaml**
   the XR to render
2. **composition.yaml**
   the pipeline to run
3. **functions.yaml**
   which function images
4. **xrd.yaml**
   schema + defaults

zsh · crossplane render

```
$ crossplane render xr.yaml composition.yaml functions.yaml --xrd xrd.yaml -r
```



Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

1. **xr.yaml**
   the XR to render
2. **composition.yaml**
   the pipeline to run
3. **functions.yaml**
   which function images
4. **xrd.yaml**
   schema + defaults

zsh · crossplane render

```
$ crossplane render xr.yaml composition.yaml functions.yaml --xrd xrd.yaml -r

kind: MyApp   checkout               # + status.conditions Ready=False
kind: Bucket  checkout
kind: Role    checkout
kind: Object  checkout-shared-logs    # Observe-only
Result: Delaying creation of "policy" because "role" is not fully ready (0 of 1)
Result: Delaying creation of "role-policy-attachment" because "role" is not fully ready (0 of 1)
```



Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

1. **xr.yaml** — the XR to render
2. **composition.yaml** — the pipeline to run
3. **functions.yaml** — which function images
4. **xrd.yaml** — schema + defaults

zsh · crossplane render

```
$ crossplane render xr.yaml composition.yaml functions.yaml --xrd xrd.yaml -r

kind: MyApp    checkout                # + status.conditions Ready=False
kind: Bucket   checkout
kind: Role     checkout
kind: Object   checkout-shared-logs    # Observe-only
Result: Delaying creation of "policy" because "role" is not fully ready (0 of 1)
Result: Delaying creation of "role-policy-attachment" because "role" is not fully ready (0 of 1)
```

-r: every function's results

The sequencer explains what it held back – before anything reaches a cluster.

Inspect it live – crossplane resource trace

On a live cluster: the whole MyApp tree, and where it is stuck, in one command

zsh · older v2 CLIs: crossplane beta trace

```
$ crossplane resource trace myapps.crossplane.best-practice.com/checkout -n team-payments
```



Inspect it live – crossplane resource trace

On a live cluster: the whole MyApp tree, and where it is stuck, in one command

zsh · older v2 CLIs: crossplane beta trace

```
$ crossplane resource trace myapps.crossplane.best-practice.com/checkout -n team-payments
```

healthy

```
NAME                                          SYNCED   READY   STATUS
MyApp/checkout (team-payments)                True     True    Available
├─ Policy/checkout (team-payments)            True     True    Available
├─ RolePolicyAttachment/checkout (team-payments)  True     True    Available
├─ Role/checkout (team-payments)              True     True    Available
├─ Object/checkout-shared-logs (team-payments)  True     True    Available
└─ Bucket/checkout (team-payments)            True     True    Available
```

Inspect it live – crossplane resource trace

On a live cluster: the whole MyApp tree, and where it is stuck, in one command

zsh · older v2 CLIs: crossplane beta trace

```
$ crossplane resource trace myapps.crossplane.best-practice.com/checkout -n team-payments
```

healthy

```
NAME                                          SYNCED   READY   STATUS
MyApp/checkout (team-payments)                True     True    Available
├─ Policy/checkout (team-payments)            True     True    Available
├─ RolePolicyAttachment/checkout (team-payments)  True     True    Available
├─ Role/checkout (team-payments)              True     True    Available
├─ Object/checkout-shared-logs (team-payments)  True     True    Available
└─ Bucket/checkout (team-payments)            True     True    Available
```

after the typo in extraPolicyActions

```
NAME                                          SYNCED   READY   STATUS
MyApp/checkout (team-payments)                True     True    Available
├─ Policy/checkout (team-payments)            False    True    ReconcileError: ...ns must be prefaced by a…
├─ RolePolicyAttachment/checkout (team-payments)  True     True    Available
├─ Role/checkout (team-payments)              True     True    Available
├─ Object/checkout-shared-logs (team-payments)  True     True    Available
└─ Bucket/checkout (team-payments)            True     True    Available
```



Inspect it live – crossplane resource trace

On a live cluster: the whole MyApp tree, and where it is stuck, in one command

zsh · older v2 CLIs: crossplane beta trace

```
$ crossplane resource trace myapps.crossplane.best-practice.com/checkout -n team-payments
```

healthy

```
NAME                                                SYNCED   READY   STATUS
MyApp/checkout (team-payments)                      True     True    Available
├─ Policy/checkout (team-payments)                  True     True    Available
├─ RolePolicyAttachment/checkout (team-payments)    True     True    Available
├─ Role/checkout (team-payments)                    True     True    Available
├─ Object/checkout-shared-logs (team-payments)      True     True    Available
└─ Bucket/checkout (team-payments)                  True     True    Available
```

One red row

The STATUS column already shows the AWS error.

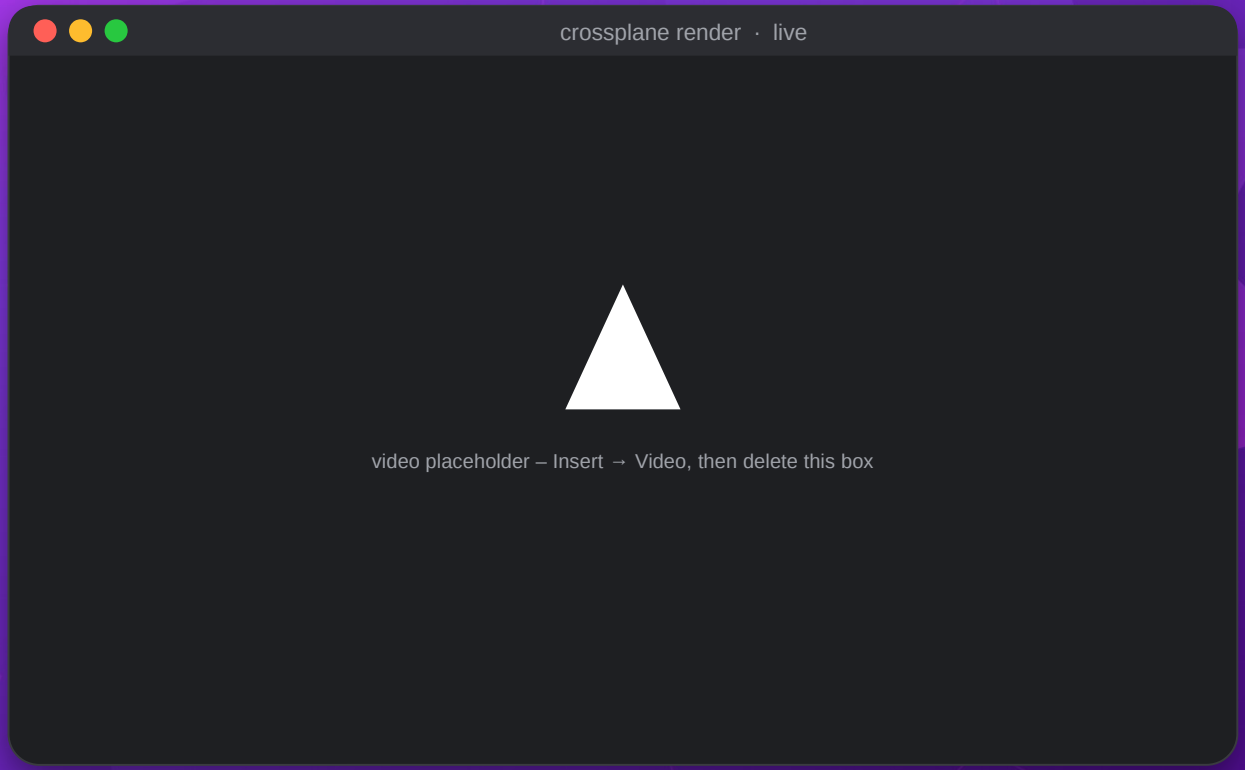
The top row still says True / True – Ready alone hides it.

after the typo in extraPolicyActions

```
NAME                                                SYNCED   READY   STATUS
MyApp/checkout (team-payments)                      True     True    Available
├─ Policy/checkout (team-payments)                  False    True    ReconcileError: ...ns must be prefaced by a…
├─ RolePolicyAttachment/checkout (team-payments)    True     True    Available
├─ Role/checkout (team-payments)                    True     True    Available
├─ Object/checkout-shared-logs (team-payments)      True     True    Available
└─ Bucket/checkout (team-payments)                  True     True    Available
```

Troubleshooting – live

From a broken Composition to the fix with crossplane render

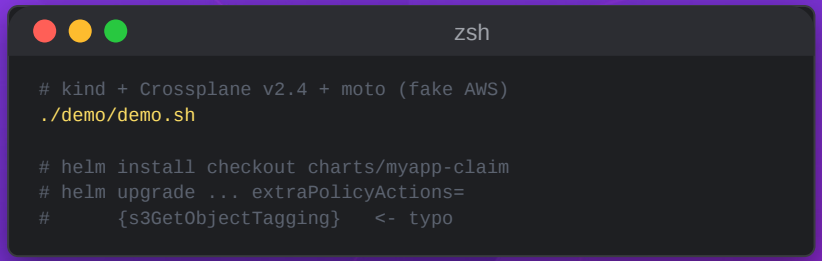


WATCH FOR

1. **Reproduce**
   render with the XR from the chart
2. **Replay**
   -o observed state from the cluster
3. **Read -r**
   which step dropped / delayed what
4. **Fix & re-render**
   before you push

Demo: composition

github.com/danielrubin1989/crossplane-composition-best-practices



Demo: composition

github.com/danielrubin1989/crossplane-composition-best-practices

zsh

```
# kind + Crossplane v2.4 + moto (fake AWS)
./demo/demo.sh

# helm install checkout charts/myapp-claim
# helm upgrade ... extraPolicyActions=
#      {s3GetObjectTagging}   <- typo
```

1. **Helm installs the platform – the raw [[ ]] is still in the cluster**



Demo: composition

github.com/danielrubin1989/crossplane-composition-best-practices

zsh

```
# kind + Crossplane v2.4 + moto (fake AWS)
./demo/demo.sh

# helm install checkout charts/myapp-claim
# helm upgrade ... extraPolicyActions=
#      {s3GetObjectTagging}   <- typo
```

1. Helm installs the platform – the raw [[ ]] is still in the cluster
2. Observe a bucket we don't own → its ARN lands in our IAM Policy



Demo: composition

github.com/danielrubin1989/crossplane-composition-best-practices

zsh

```
# kind + Crossplane v2.4 + moto (fake AWS)
./demo/demo.sh

# helm install checkout charts/myapp-claim
# helm upgrade ... extraPolicyActions=
#      {s3GetObjectTagging}   <- typo
```

1. Helm installs the platform – the raw [[ ]] is still in the cluster
2. Observe a bucket we don't own → its ARN lands in our IAM Policy
3. Sequencer events, then MyApp Ready only after all 5 children



Demo: composition

github.com/danielrubin1989/crossplane-composition-best-practices

zsh

```
# kind + Crossplane v2.4 + moto (fake AWS)
./demo/demo.sh

# helm install checkout charts/myapp-claim
# helm upgrade ... extraPolicyActions=
#      {s3GetObjectTagging}   <- typo
```

1. Helm installs the platform – the raw [[ ]] is still in the cluster
2. Observe a bucket we don't own → its ARN lands in our IAM Policy
3. Sequencer events, then MyApp Ready only after all 5 children
4. Typo → HEALTHY=False with the AWS message, READY still True



Demo: composition

github.com/danielrubin1989/crossplane-composition-best-practices

```
zsh

# kind + Crossplane v2.4 + moto (fake AWS)
./demo/demo.sh

# helm install checkout charts/myapp-claim
# helm upgrade ... extraPolicyActions=
#      {s3GetObjectTagging}   <- typo
```

1. Helm installs the platform – the raw [[ ]] is still in the cluster
2. Observe a bucket we don't own → its ARN lands in our IAM Policy
3. Sequencer events, then MyApp Ready only after all 5 children
4. Typo → HEALTHY=False with the AWS message, READY still True
5. crossplane render: create, wire and fail – offline

